КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ імені Тараса Шевченка ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра програмних систем і технологій

Дисципліна

«КОНСТРУЮВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ»

Лабораторна робота № 2

«Базова реалізація, Парсинг та Моделі даних»

Виконав здобувач:	Роман ПЛАХОТА	**Перевірив**:	Павло ШАБАТІН
Група, підгрупа	ІПЗ-34, 10.	Дата перевірки	

2026

Варіант 5: Симулятор віртуальної файлової системи (VFS Shell)

Мета: Набути практичних навичок у:

- перетворенні архітектурного проєкту у працюючий програмний код;
- реалізації читання та синтаксичного поділу текстових даних;
- побудові складних структур даних в оперативній пам'яті (Trees, Graphs, Queues);
- дотриманні принципів Separation of Concerns (розділення парсингу, збереження стану та виводу);
- написанні коду, що відповідає обраним стандартам (Clean Code, проходження перевірки лінтером).

Умова завдання:

Що реалізувати: Зчитування скрипту script.sh та парсинг команд.

Внутрішня модель: Побудова N-арного дерева або іншої структури даних для репрезентації папок та файлів.

Очікуваний результат роботи програми: Програма виконує команди mkdir, touch, cd у пам'яті. При зустрічі команди ls - виводить поточний вміст віртуальної директорії у консоль.

Приклад вхідних даних (script.sh):

```
# Ініціалізація диска
mkfs 4096

# Створення базової структури
mkdir /etc
mkdir /home
mkdir /home/admin
mkdir /home/guest
mkdir /var
mkdir /var/log

# Створення файлів
touch /etc/config.json "{\"os\": \"vfs\", \"version\": 1.0}"
touch /home/admin/readme.txt "Welcome to the system."
touch /var/log/syslog.log "System booted successfully."
touch /var/log/auth.log "Admin logged in."

# Навігація та налаштування прав
cd /home/admin
```

```
chmod 700 readme.txt
cd /var/log
chmod 644 syslog.log

# Вивід вмісту для перевірки
ls /
ls /var/log
cat /etc/config.json
```

Хід роботи

1) Модуль моделей (src/models.py)

Цей модуль визначає базову структуру даних проєкту. Він реалізує патерн Composite, що дозволяє однаково працювати як з окремими файлами, так і з директоріями.

```
class INode(ABC):
    """
    Abstract base class for all file system elements.
    """

    def __init__(
        self, name: str, parent: Optional["Directory"] = None, permissions: int =
0o777
    ) -> None:
        self.name: str = name
        self.parent: Optional["Directory"] = parent
        self.permissions: int = permissions

    @abstractmethod
    def get_size(self) -> int:
        """Returns the size of the node in bytes."""
        pass

    def get_path(self) -> str:
        """
        Recursively builds the absolute path to the current node.
        """
        if self.parent is None:
            return self.name if self.name == "/" else f"/{self.name}"

        parent_path = self.parent.get_path()
        if parent_path == "/":
            return f"/{self.name}"
        return f"{parent_path}/{self.name}"


class File(INode):
    """
```

```python
    A class representing a text file in VFS.
    """

    def __init__(
        self,
        name: str,
        content: str = "",
        parent: Optional["Directory"] = None,
        permissions: int = 0o644,
    ) -> None:
        # Files have default permissions of 644 (rw-r--r--)
        super().__init__(name, parent, permissions)
        self.content: str = content

    def get_size(self) -> int:
        """
        Returns the file size.
        For MVP, we consider 1 character = 1 byte.
        """
        return len(self.content)


class Directory(INode):
    """
    A class representing a directory in VFS.
    Contains a dictionary of child elements for fast O(1) access.
    """

    def __init__(
        self, name: str, parent: Optional["Directory"] = None, permissions: int =
0o755
    ) -> None:
        # Directories have default permissions of 755 (rwxr-xr-x)
        super().__init__(name, parent, permissions)
        self.children: Dict[str, INode] = {}

    def get_size(self) -> int:
        """
        Recursively calculates the total size of all embedded files and folders.
        """
        return sum(child.get_size() for child in self.children.values())

    def add_child(self, node: INode) -> None:
        """Adds new node to directory."""
        if node.name in self.children:
            raise ValueError(f"Element with name '{node.name}' already exists.")
        node.parent = self
        self.children[node.name] = node

    def remove_child(self, name: str) -> None:
        """Removes node from directory based on name."""
        if name not in self.children:
            raise KeyError(f"Element '{name}' not found")
```

```
        del self.children[name]

    def get_child(self, name: str) -> Optional[INode]:
        """Returns the child node by name or None if it does not exist."""
        return self.children.get(name)
```

Абстрактний клас INode містить метадані такі як права доступу, власник, посилання на батьківський вузол. Клас Directory використовує хеш-таблицю (dict) для зберігання дочірніх об'єктів, що забезпечує швидкість пошуку за ім'ям. Метод get_path() рекурсивно піднімається до кореня для побудови повного шляху до об'єкта.

2) Модуль команд (src/commands.py)

Реалізує патерн Command. Кожна операція у системі — це окремий клас, що інкапсулює логіку конкретної дії.

```
class ICommand(ABC):
    """Abstract interface for all system commands."""

    @abstractmethod
    def execute(self, context: VFSContext) -> CommandResult:
        """
        Does action in given context.
        Returns data to print or None.
        """
        pass


class MkfsCommand(ICommand):
    def __init__(self, max_size: int) -> None:
        self.max_size = max_size

    def execute(self, context: VFSContext) -> str:
        context.max_size = self.max_size
        return f"The virtual file system has been initialized. Size:
{self.max_size} bytes."


class MkdirCommand(ICommand):
    def __init__(self, path: str) -> None:
        self.path = path

    def execute(self, context: VFSContext) -> None:
        parent, name = get_parent_and_name(context, self.path)
        if not parent:
            raise ValueError(f"mkdir: cannot create '{self.path}': No such file or
directory")
```

```
        new_dir = Directory(name=name)
        parent.add_child(new_dir)
```

Всі команди наслідуються від інтерфейсу ICommand. Метод execute приймає VFSContext, що дає команді доступ до файлової системи, але обмежує її вплив лише цим об'єктом. Тип повернення CommandResult гарантує, що форматор зможе коректно відобразити дані.

3) Модуль парсера (src/parser.py)

Цей модуль виконує роль текстового аналізатора. Він перетворює сирий рядок вводу на об'єкт команди, використовуючи таблицю переходів.

```
class InputParser:
    """
    Класс для перетворення вхідних рядків у об'єкти команд.
    """

    @staticmethod
    def parse(raw_input: str) -> Optional[ICommand]:
        """
        Converts a string to a command object.
        Returns None if the string is empty or is a comment.
        """
        line = raw_input.strip()

        # check for comment or empty line
        if not line or line.startswith("#"):
            return None

        # Split line on tokens
        tokens = shlex.split(line)

        if not tokens:
            return None

        command_name = tokens[0].lower()
        args = tokens[1:]

        # Define factories for each command
        # Each lambda takes a list of arguments and returns a command object
        factories: Dict[str, Callable[[list[str]], ICommand]] = {
            "mkfs": lambda a: MkfsCommand(max_size=int(a[0])),
            "mkdir": lambda a: MkdirCommand(path=a[0]),
            "touch": lambda a: TouchCommand(path=a[0], content=a[1] if len(a) > 1
else ""),
            "cd": lambda a: CdCommand(path=a[0]),
            "chmod": lambda a: ChmodCommand(permissions=int(a[0], 8), path=a[1]),
            "ls": lambda a: LsCommand(path=a[0] if a else ""),
            "cat": lambda a: CatCommand(path=a[0]),
```

```
            "cls": lambda _: ClsCommand(),
            "clear": lambda _: ClsCommand(),
            "exit": lambda _: ExitCommand(),
            "quit": lambda _: ExitCommand(),
        }

        try:
            if command_name not in factories:
                raise ValueError(f"Command '{command_name}' is not supported")

            return factories[command_name](args)
```

Використання бібліотеки shlex дозволяє коректно обробляти аргументи в лапках, наприклад, вміст файлу. Словник factories з лямбда-функціями дозволяє уникнути великої кількості операторів if-else (помилка PLR0911), роблячи код легко розширюваним.

4) Модуль безпеки (src/security.py)

Відповідає за перевірку прав доступу користувача до об'єктів файлової системи.

```
class SecurityEngine:
    # Constants for checks
    READ = 4
    WRITE = 2
    EXECUTE = 1

    def has_access(self, node: INode, user: str, required_access: int) -> bool:
        """
        Checks whether the user has access to the node.
        MVP: admin has access to everything, others — according to permissions.
        """
        if user == "admin":
            return True

        # Get permissions (e.g., 755 -> last digit 5 for ‘others’)
        # Simplified approach: take the last digit (rights for everyone)
        others_permissions = node.permissions % 10

        # Checking with bitwise 'AND'
        return (others_permissions & required_access) == required_access
```

Модуль використовує POSIX-подібну модель прав. Права зберігаються як ціле число (наприклад, 755), а перевірка виконується за допомогою побітових операцій. Це забезпечує високу продуктивність та імітує поведінку реальних Unix-систем.

5) Модуль виконання та оркестрації (src/engine.py та main.py)

ExecutionEngine координує роботу команди, безпеки та форматування. main.py виступає в ролі контролера.

```
class ExecutionEngine:
```

```
    def __init__(self, context: VFSContext, formatter: OutputFormatter) -> None:
        self.context = context
        self.formatter = formatter
        self.security = SecurityEngine()

    def execute(self, command: ICommand, raw_line: str) -> None:
        try:
            # Identifying command name
            cmd_name = raw_line.split(maxsplit=1)[0].lower()

            required_access = self.security.get_required_access(cmd_name)

            # security check
            if not self.security.has_access(
                self.context.current_directory, self.context.current_user,
required_access
            ):
                raise PermissionError(f"Access denied for '{cmd_name}'")

            # visualization and execution
            self.formatter.render_command_echo(raw_line)
            result = command.execute(self.context)
            self.formatter.render_result(result)

        except Exception as e:
            self.formatter.render_error(str(e))
```

Енджин реалізує централізовану обробку помилок та перевірку прав. Такий підхід гарантує, що жодна команда не буде виконана без попередньої валідації безпеки. Модуль main.py забезпечує два режими роботи: інтерактивний (CLI) та пакетний (виконання .sh скриптів).

6) Модуль представлення (src/formatter.py)

Відповідає за візуалізацію результатів у терміналі.

```
class OutputFormatter:
    def render_comment(self, text: str) -> None:
        """Prints comments with grey color."""
        print(f"\n\033[90m{text}\033[0m")

    def render_command_echo(self, text: str) -> None:
        """Echoes command that is currently running."""
        print(f"\n> {text}")

    def render_result(self, result: CommandResult) -> None:
        # If command returns true then don't print anything
        if result is True:
            return
        # If the command returns nothing (mkdir, touch, cd), print OK
        if result is None:
            print("  \033[92m[OK]\033[0m")
            return
```

```
        if isinstance(result, list):
            self._render_ls(result)
        else:
            # For outputting text (cat) or messages
            print(f"  \033[94m{result}\033[0m")
```

Модуль інкапсулює всі знання про ANSI-коди кольорів та правила відступів. Це дозволяє змінювати зовнішній вигляд програми, наприклад, додати табличний вивід або веб-інтерфейс, не змінюючи логіку самих команд чи ядра системи.

Код програми:

Код програми даного застостунку надано в Github-репозиторії за посиланням: https://github.com/xanaxhol1c/vfs-shell-python

Результат роботи:

Програма успішно відпрацьовує базові сценарії як для **script.sh** так і для **live режиму.**

Script.sh

```
PS C:\Vscode\Repos\vfs-shell-python> poetry run python main.py script.sh
Running script: script.sh

# Ініціалізація диска

> mkfs 4096
  The virtual file system has been initialized. Size: 4096 bytes.

# Створення базової структури

> mkdir /etc
  [OK]

> mkdir /home
  [OK]

> mkdir /home/admin
  [OK]

> mkdir /home/guest
  [OK]

> mkdir /var
  [OK]

> mkdir /var/log
```

```
# Створення файлів

> touch /etc/config.json "{\"os\": \"vfs\", \"version\": 1.0}"
  [OK]

> touch /home/admin/readme.txt "Welcome to the system."
  [OK]

> touch /var/log/syslog.log "System booted successfully."
  [OK]

> touch /var/log/auth.log "Admin logged in."
  [OK]

# Навігація та налаштування прав

> cd /home/admin

> cd /var/log
  [OK]

> chmod 644 syslog.log
  [OK]
```

```
# Вивід вмісту для перевірки

> ls /
  TYPE   PERM   SIZE       NAME
  ----------------------------------------
  DIR    755    29 B       etc
  DIR    755    22 B       home
  DIR    755    43 B       var

> ls /var/log
  TYPE   PERM   SIZE       NAME
  ----------------------------------------
  FILE   644    27 B       syslog.log
  FILE   644    16 B       auth.log

> cat /etc/config.json
  {"os": "vfs", "version": 1.0}
```

Live режим

```
PS C:\Vscode\Repos\vfs-shell-python> poetry run python main.py
VFS Interactive Shell (MVP)
Type 'exit' or 'quit' to exit.

vfs:/$ mkfs 2048

> mkfs 2048
  The virtual file system has been initialized. Size: 2048 bytes.
vfs:/$ touch file.txt "its life mode in vfs shell"

> touch file.txt "its life mode in vfs shell"
  [OK]
vfs:/$ ls

> ls
  TYPE   PERM   SIZE       NAME
  ----------------------------------------
  FILE   644    26 B       file.txt
vfs:/$ cat file.txt

> cat file.txt
  its life mode in vfs shell
vfs:/$
```

Висновок:

У ході виконання лабораторної роботи було створено консольний застосунок на мові програмування Python, який реалізує симулятор віртуальної файлової системи з підтримкою багатокористувацького доступу та пакетного виконання скриптів. Було розроблено модульну архітектуру програми за принципами Clean Architecture, що забезпечило чітке розділення відповідальності між парсером команд, оркестратором виконання, модулем безпеки та засобами візуалізації даних.

В основі системи реалізовано ієрархічну структуру даних у вигляді N-арного дерева з використанням патерна Composite, що дозволило поліморфно оперувати об'єктами файлів та директорій. Програма підтримує повноцінний життєвий цикл роботи з файловою системою: від ініціалізації дискового простору до маніпуляції структурою вузлів та вмістом файлів. Окрему увагу було приділено системі безпеки, яка базується на POSIX-подібній моделі прав доступу та використовує побітові операції для валідації дозволів користувачів.